engineers. Figure 2-6 shows three *continuous* waveforms and their *pdfs*. If these were *discrete signals*, signified by changing the horizontal axis labeling to "sample number," *pmfs* would be used.

A problem occurs in calculating the histogram when the number of levels each sample can take on is much larger than the number of samples in the signal. This is always true for signals represented in *floating point* notation, where each sample is stored as a fractional value. For example, integer representation might require the sample value to be 3 or 4, while floating point allows millions of possible fractional values *between* 3 and 4. The previously described approach for calculating the histogram involves counting the number of samples that have each of the possible quantization levels. This is not possible with floating point data because there are *billions* of possible levels that would have to be taken into account. Even worse, nearly all of these possible levels would have no samples that correspond to them. For example, imagine a 10,000 sample signal, with each sample having one billion possible values. The conventional histogram would consist of one billion data points, with all but about 10,000 of them having a value of zero.

The solution to these problems is a technique called **binning**. This is done by arbitrarily selecting the length of the histogram to be some convenient number, such as 1000 points, often called **bins**. The value of each bin represent the total number of samples in the signal that have a value within a *certain range*. For example, imagine a floating point signal that contains values from 0.0 to 10.0, and a histogram with 1000 bins. Bin 0 in the histogram is the number of samples in the signal with a value between 0 and 0.01, bin 1 is the number of samples with a value between 0.01 and 0.02, and so forth, up to bin 999 containing the number of samples with a value between 9.99 and 10.0. Table 2-4 presents a program for calculating a binned histogram in this manner.

```
100 'CALCULATION OF BINNED HISTOGRAM
110 '
120 DIM X[25000]                    'X[0] to X[25000] holds the floating point signal,
130 '                               'with each sample being in the range: 0.0 to 10.0
140 DIM H%[999]                     'H%[0] to H%[999] holds the binned histogram
150 '
160 FOR I% = 0 TO 999               'Zero the binned histogram for use as an accumulator
170   H%[I%] = 0
180 NEXT I%
190 '
200 GOSUB XXXX                      'Mythical subroutine that loads the signal into X%[ ]
210 '
220 FOR I% = 0 TO 25000  'Calculate the binned histogram for 25001 points
230   BINNUM% = INT( X[I%] * .01 )
240   H%[ BINNUM%] = H%[ BINNUM%] + 1
250 NEXT I%
260 '
270 END
```

TABLE 2-4